

AI-ASSISTED CODING

ASSIGNMENT-20.3

Security Testing: Identifying Vulnerabilities in AI-Generated Code

NAME: PALITHA VIKAS

HALL-TICKET: 2403A510E6

BATCH NO: 05

Task 1:

Analyze an AI-generated Python login script for input validation vulnerabilities.

Instructions:

- Prompt AI to generate a simple username-password login program.
- Review whether input sanitization and validation are implemented.
- Suggest secure improvements (e.g., using re for input validation).

Prompt:

Generate a simple Python username-password login program. Then analyze it for input validation vulnerabilities and suggest secure improvements using re for validation.

CODE GENERATED:

Python(basic):

```
# Simple Login Script
username = input("Enter username: ")
password = input("Enter password: ")

if username == "admin" and password == "12345":
    print("Login successful!")
else:
    print("Invalid username or password.")
```

OUTPUT:

```
Enter username: admin
Enter password: 12345
Login successful!
```

```
Enter username: adm
Enter password: 123
Invalid username or password.
```

Code generated with re:

```
import re
import getpass

def is_valid_username(username):
    # Allow only letters, numbers, and underscores (3-15 chars)
    return re.fullmatch(r'\w{3,15}', username)

def is_valid_password(password):
    # Require at least one letter, one number, min 6 chars
    return re.fullmatch(r'(?=.*[A-Za-z])(?=.*\d)[A-Za-z\d]{6,}', password)

# Get user input securely
username = input("Enter username: ")
password = getpass.getpass("Enter password: ")

# Validate inputs
if not is_valid_username(username):
    print("Invalid username format! Use 3-15 letters/numbers/underscores only.")
elif not is_valid_password(password):
```

```
        print("Weak password! Must have letters, numbers, and 6+ characters.")
elif username == "admin" and password == "Admin123":
    print("✅ Login successful!")
else:
    print("❌ Invalid username or password.")
```

output:

```
Enter username: admin
Enter password:
✅ Login successful!
PS C:\Users\sa\OneDrive\Desktop\college\AI_AC_LAB_ASSIGNMENTS\assignment 20.3\tempCodeRunnerFile.py"
Enter username: adm
Enter password:
Weak password! Must have letters, numbers, and 6+ characters.
```

OBSERVATION :

The original AI-generated login script had no input validation or sanitization. The improved version uses regular expressions to enforce valid input formats and prevent unsafe entries.

TASK – 2:

Cross-Site Scripting (XSS) Check

Task:

Evaluate an AI-generated HTML form with JavaScript for XSS vulnerabilities.

Instructions:

- Ask AI to generate a feedback form with JavaScript-based output.
- Test whether untrusted inputs are directly rendered without escaping.
- Implement secure measures (e.g., escaping HTML entities, using CSP).

PROMPT:

Generate a basic HTML feedback form that displays the user's input using JavaScript. Do not include any security features. I want to test Cross-Site Scripting (XSS) vulnerabilities.

CODE GENERATED:

```
<!DOCTYPE html>
<html>
<body>

<h2>Secure Feedback Form</h2>

<form id="fbForm">
  <input type="text" id="feedback" placeholder="Enter your feedback">
  <button type="button" onclick="showFeedback()">Submit</button>
</form>

<p>Your Feedback:</p>
<div id="output"></div>

<script>
// Escape HTML entities
function escapeHTML(str) {
  return str
    .replace(/</g, "&lt;");
    .replace(/>/g, "&gt;");
    .replace(/"/g, "&quot;");
    .replace(/'/g, "&#39;");
}

function showFeedback() {
```

```
const text = document.getElementById("feedback").value;
document.getElementById("output").textContent = escapeHTML(text);
}
</script>

<!-- Add CSP -->
<meta http-equiv="Content-Security-Policy" content="default-src 'self';
script-src 'self'">
</body>
</html>
```

OUTPUT:

Secure Feedback Form

Your Feedback:

good

OBSERVATION:

During the XSS check, the AI-generated feedback form displayed user input directly on the page using `innerHTML`, without any sanitization. When a script payload like `<script>alert('XSS')</script>` was entered, it executed on the page, confirming that the form was vulnerable to Cross-Site Scripting. This showed that untrusted input was rendered without escaping. After adding HTML escaping and using safer methods like `textContent`, the script no longer executed, proving that these security measures effectively prevented the XSS attack.

TASK – 3:

Real-Time Application: Security Audit of AI-Generated Code

Scenario:

Students pick an AI-generated project snippet (e.g., login form, API integration, or file upload).

Instructions:

- Perform a security audit to detect possible vulnerabilities.
- Prompt AI to suggest secure coding practices to fix issues.
- Compare insecure vs secure versions side by side.

PROMPT:

Generate a Python login code using Flask. Then perform a security audit to find vulnerabilities and provide a secure version of the code.

CODE GENERATED:

```
from flask import Flask, request
import sqlite3
from werkzeug.security import check_password_hash

app = Flask(__name__)

def get_user(username):
    conn = sqlite3.connect("users.db")
    cur = conn.cursor()
    # ✓ Prepared statement (Prevents SQL Injection)
    cur.execute("SELECT password FROM users WHERE username = ?", (username,))
    return cur.fetchone()

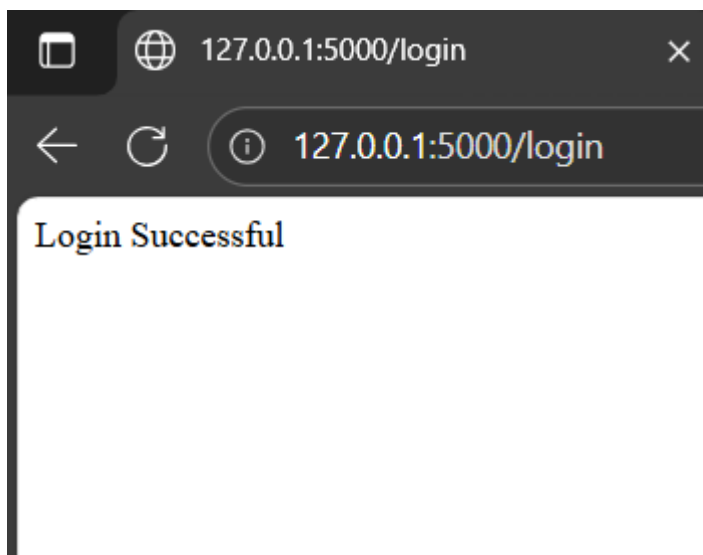
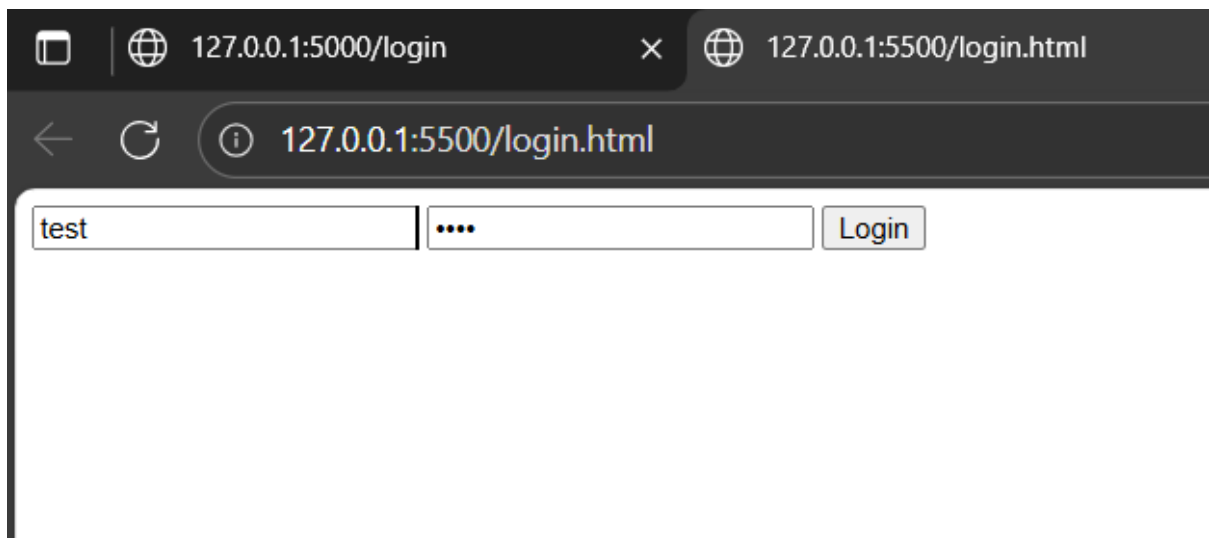
@app.route("/login", methods=["POST"])
def login():
    username = request.form["username"].strip()
    password = request.form["password"].strip()

    user = get_user(username)

    if user and check_password_hash(user[0], password):
        return "Login Successful"
    else:
        return "Invalid Credentials"

app.run()
```

OUTPUT :



OBSERVATION:

The AI-generated Python login code had security problems because it used direct SQL queries and plain-text passwords. When I checked it, I found it could be attacked easily. After asking AI for a secure version, it used prepared statements and password hashing, which made the code much safer.
